



Alireza Garmsiri

Evolutionary Computing

HW1 : N Queen



Introduction

The N-Queens problem is a classic combinatorial optimization challenge in which the objective is to place N queens on an $N \times N$ chessboard such that no two queens threaten each other. This means that no two queens can share the same row, column, or diagonal.

As the value of N increases, the problem becomes more complex, making brute-force solutions infeasible due to the exponential increase in possible board configurations. To efficiently tackle this problem, various approaches such as backtracking, constraint satisfaction, and heuristic algorithms have been proposed. One of the popular heuristic approaches is **Genetic Algorithms (GAs)**, which are based on the principles of natural selection and evolution.

This report covers the implementation and results of solving the N-Queens problem using a genetic algorithm. The primary goal is to demonstrate the capabilities of GAs to find valid solutions through evolutionary processes, including crossover, mutation, and selection. The algorithm also explores various parameter configurations, such as mutation and crossover rates, as well as different strategies (Elitism vs. Generational)

Code

Population Representation: Each individual (potential solution) is represented as a list of integers, where each integer denotes the position of a queen in a particular column. For instance, in an 8-Queens problem, an individual could be represented as [4, 2, 7, 3, 6, 8, 5, 1], where the value in each position of the list corresponds to the row number of the queen in that column.

Fitness Function: The fitness of an individual is determined by counting the number of queen conflicts. A conflict occurs if two queens are on the same row or diagonal. The goal is to minimize the number of conflicts, with a fitness value of zero indicating a solution where no queens threaten each other.

Selection: The selection process chooses individuals from the population to serve as parents for the next generation. The algorithm uses a "tournament selection" strategy, where a small subset of individuals is randomly selected from the population, and the best individuals (those with the fewest conflicts) are chosen as parents.

Crossover (Recombination): In the crossover step, two parent individuals are combined to create new offspring. The algorithm implements a single-point crossover where a random cut is made in the parents' lists, and the offspring inherit the first part from one parent and the second part from the other.

Mutation: Mutation introduces random changes in an individual to maintain diversity in the population. This is crucial for avoiding local optima and ensuring the algorithm explores a broad range of solutions. In the N-Queens problem, mutation involves randomly swapping two queens in an individual's list.

Elitism vs. Generational Strategies: The algorithm supports two strategies for population evolution:

Elitism: The best individual from each generation is preserved and carried over to the next generation to ensure that the best solution so far is not lost.

Generational: A completely new population is generated in each generation without preserving the best individual from the previous generation.

Result

Higher mutation rates (e.g., 1.0) tend to introduce more diversity in the population, but may lead to slower convergence as good solutions can be disrupted.

Higher crossover rates (e.g., 1.0) allow for faster recombination of good traits and tend to speed up the convergence process.

The elitism strategy often outperforms the generational strategy because it ensures that the best solution found so far is preserved and improved upon, while the generational strategy may lose good individuals from one generation to the next.

Larger board sizes (e.g., $N=20$) naturally take more generations to solve due to the increased complexity of the search space.

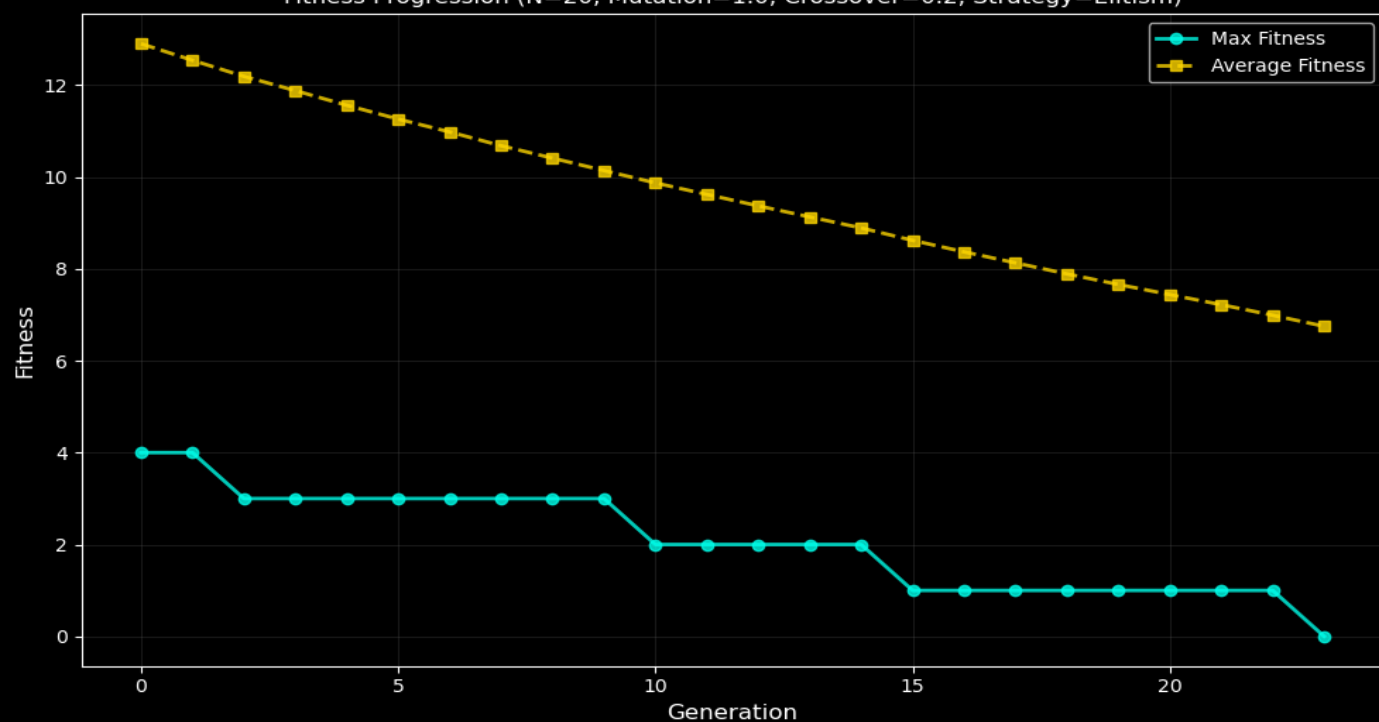
Running GA for N=20, Mutation Rate=1.0, Crossover Rate=0.2, Strategy=Elitism

```

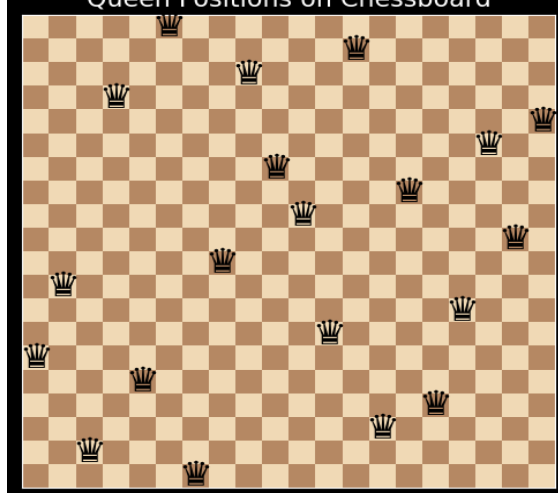
0 : Best F : 4 | AVG : 12.9 | [6, 13, 9, 4, 18, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 16, 4] | len : 21
1 : Best F : 4 | AVG : 12.54 | [6, 13, 9, 4, 18, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 16, 4] | len : 21
2 : Best F : 3 | AVG : 12.19 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
3 : Best F : 3 | AVG : 11.88 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
4 : Best F : 3 | AVG : 11.56 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
5 : Best F : 3 | AVG : 11.26 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
6 : Best F : 3 | AVG : 10.98 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
7 : Best F : 3 | AVG : 10.68 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
8 : Best F : 3 | AVG : 10.41 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
9 : Best F : 3 | AVG : 10.14 | [6, 13, 9, 4, 16, 3, 15, 10, 11, 19, 12, 5, 17, 20, 1, 8, 2, 14, 7, 18, 3] | len : 21
10 : Best F : 2 | AVG : 9.87 | [6, 13, 9, 4, 20, 16, 10, 15, 11, 19, 2, 5, 17, 12, 1, 8, 18, 14, 3, 7, 2] | len : 21
11 : Best F : 2 | AVG : 9.62 | [6, 13, 9, 4, 20, 16, 10, 15, 11, 19, 2, 5, 17, 12, 1, 8, 18, 14, 3, 7, 2] | len : 21
12 : Best F : 2 | AVG : 9.37 | [6, 13, 9, 4, 20, 16, 10, 15, 11, 19, 2, 5, 17, 12, 1, 8, 18, 14, 3, 7, 2] | len : 21
13 : Best F : 2 | AVG : 9.13 | [6, 13, 9, 4, 20, 16, 10, 15, 11, 19, 2, 5, 17, 12, 1, 8, 18, 14, 3, 7, 2] | len : 21
14 : Best F : 2 | AVG : 8.89 | [6, 13, 9, 4, 20, 16, 10, 15, 11, 19, 2, 5, 17, 12, 1, 8, 18, 14, 3, 7, 2] | len : 21
15 : Best F : 1 | AVG : 8.62 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
16 : Best F : 1 | AVG : 8.37 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
17 : Best F : 1 | AVG : 8.13 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
18 : Best F : 1 | AVG : 7.89 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
19 : Best F : 1 | AVG : 7.66 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
20 : Best F : 1 | AVG : 7.44 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
21 : Best F : 1 | AVG : 7.22 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
22 : Best F : 1 | AVG : 6.99 | [6, 13, 9, 4, 20, 16, 2, 15, 11, 19, 10, 5, 17, 12, 1, 8, 18, 14, 3, 7, 1] | len : 21
23 : Best F : 0 | AVG : 6.75 | [6, 13, 9, 4, 20, 18, 10, 15, 11, 19, 8, 2, 17, 12, 1, 5, 16, 14, 3, 7, 0] | len : 21
Solution found at generation 23, Best Gene: [6, 13, 9, 4, 20, 18, 10, 15, 11, 19, 8, 2, 17, 12, 1, 5, 16, 14, 3, 7]

```

Fitness Progression (N=20, Mutation=1.0, Crossover=0.2, Strategy=Elitism)



Queen Positions on Chessboard

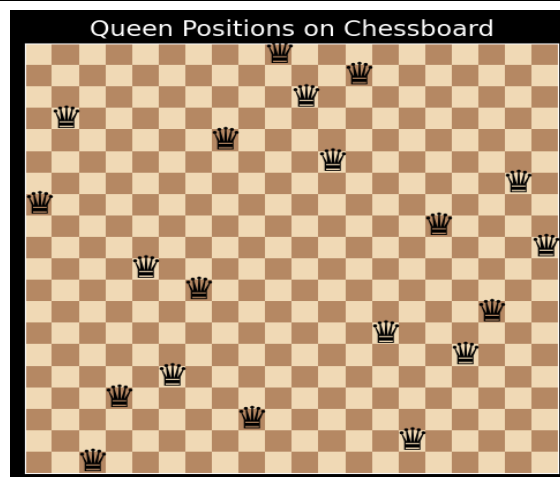


Running GA for N=20, Mutation Rate=0.5, Crossover Rate=0.5, Strategy=Generational

```

0 : Best F : 5 | AVG : 10.95 | [17, 13, 9, 2, 8, 12, 19, 1, 16, 20, 11, 7, 6, 18, 14, 5, 3, 10, 4, 15, 5] | len : 21
1 : Best F : 4 | AVG : 9.87 | [17, 13, 9, 2, 8, 14, 19, 1, 16, 20, 11, 7, 6, 18, 12, 5, 3, 10, 4, 15, 4] | len : 21
2 : Best F : 2 | AVG : 8.95 | [17, 13, 9, 2, 8, 14, 19, 1, 16, 20, 11, 3, 6, 18, 12, 5, 7, 10, 4, 15, 2] | len : 21
3 : Best F : 2 | AVG : 8.88 | [17, 13, 9, 2, 8, 14, 19, 1, 16, 20, 11, 3, 6, 18, 12, 5, 7, 10, 4, 15, 2] | len : 21
4 : Best F : 2 | AVG : 8.31 | [17, 13, 9, 2, 8, 14, 19, 1, 16, 20, 11, 3, 6, 18, 12, 5, 7, 10, 4, 15, 2] | len : 21
5 : Best F : 3 | AVG : 8.17 | [10, 13, 9, 2, 8, 14, 19, 1, 16, 20, 11, 7, 6, 18, 12, 5, 3, 17, 15, 4, 3] | len : 21
6 : Best F : 3 | AVG : 8.1 | [14, 13, 9, 6, 8, 2, 19, 1, 16, 20, 11, 7, 17, 4, 12, 5, 3, 10, 18, 15, 3] | len : 21
7 : Best F : 3 | AVG : 7.25 | [14, 13, 9, 6, 8, 2, 19, 1, 16, 20, 11, 7, 17, 4, 12, 5, 3, 10, 18, 15, 3] | len : 21
8 : Best F : 3 | AVG : 7.6 | [13, 5, 19, 11, 4, 17, 20, 9, 7, 18, 14, 6, 8, 16, 3, 10, 2, 15, 1, 12, 3] | len : 21
9 : Best F : 3 | AVG : 7.65 | [14, 13, 9, 6, 8, 2, 19, 1, 16, 18, 11, 7, 17, 4, 12, 5, 3, 10, 20, 15, 3] | len : 21
10 : Best F : 3 | AVG : 7.09 | [14, 13, 9, 6, 8, 2, 19, 1, 16, 18, 11, 7, 17, 4, 12, 5, 3, 10, 20, 15, 3] | len : 21
11 : Best F : 3 | AVG : 7.24 | [14, 13, 9, 6, 8, 2, 19, 1, 16, 20, 11, 7, 17, 4, 12, 5, 3, 10, 18, 15, 3] | len : 21
12 : Best F : 4 | AVG : 7.7 | [17, 13, 4, 2, 8, 15, 19, 1, 16, 12, 5, 20, 6, 18, 14, 7, 3, 9, 11, 10, 4] | len : 21
13 : Best F : 3 | AVG : 8.04 | [6, 13, 9, 14, 8, 2, 19, 1, 7, 18, 11, 20, 17, 4, 16, 5, 3, 12, 10, 15, 3] | len : 21
14 : Best F : 3 | AVG : 7.09 | [6, 13, 9, 14, 8, 2, 19, 1, 7, 18, 11, 20, 17, 4, 16, 5, 3, 12, 10, 15, 3] | len : 21
15 : Best F : 4 | AVG : 7.3 | [10, 13, 9, 2, 8, 12, 19, 1, 16, 20, 5, 7, 14, 18, 6, 11, 4, 17, 15, 3, 4] | len : 21
16 : Best F : 4 | AVG : 7.24 | [10, 13, 9, 2, 8, 12, 19, 11, 16, 20, 5, 7, 14, 18, 6, 1, 4, 17, 15, 3, 4] | len : 21
17 : Best F : 4 | AVG : 7.67 | [17, 4, 9, 2, 6, 20, 18, 1, 16, 7, 5, 12, 8, 19, 14, 15, 13, 10, 3, 11, 4] | len : 21
18 : Best F : 3 | AVG : 6.7 | [17, 3, 9, 2, 6, 20, 18, 1, 12, 7, 5, 15, 8, 19, 14, 16, 13, 10, 4, 11, 3] | len : 21
19 : Best F : 3 | AVG : 7.47 | [10, 13, 9, 2, 8, 12, 19, 1, 16, 20, 5, 7, 14, 18, 6, 11, 3, 17, 15, 4, 3] | len : 21
20 : Best F : 3 | AVG : 7.29 | [10, 13, 9, 2, 8, 12, 19, 1, 16, 20, 5, 7, 14, 18, 6, 3, 17, 15, 4, 11, 3] | len : 21
21 : Best F : 4 | AVG : 7.38 | [10, 13, 4, 2, 8, 12, 19, 1, 16, 20, 5, 7, 14, 18, 6, 11, 9, 17, 15, 3, 4] | len : 21
22 : Best F : 4 | AVG : 7.08 | [10, 13, 9, 2, 8, 12, 19, 1, 16, 20, 5, 14, 7, 18, 6, 11, 4, 17, 15, 3, 4] | len : 21
23 : Best F : 3 | AVG : 7.81 | [17, 5, 9, 2, 6, 14, 20, 1, 16, 7, 3, 12, 8, 18, 19, 15, 13, 10, 4, 11, 3] | len : 21
...
119 : Best F : 1 | AVG : 3.05 | [10, 13, 11, 2, 8, 12, 19, 1, 16, 20, 5, 7, 18, 14, 6, 4, 9, 17, 15, 3, 1] | len : 21
120 : Best F : 1 | AVG : 2.64 | [10, 13, 11, 2, 8, 12, 19, 1, 16, 20, 5, 7, 18, 14, 6, 4, 9, 17, 15, 3, 1] | len : 21
121 : Best F : 0 | AVG : 2.54 | [10, 13, 11, 2, 8, 12, 19, 1, 16, 20, 5, 7, 18, 14, 17, 6, 4, 9, 15, 3, 0] | len : 21
Solution found at generation 121, Best Gene: [10, 13, 11, 2, 8, 12, 19, 1, 16, 20, 5, 7, 18, 14, 17, 6, 4, 9, 15, 3]

```



Conclusion

The genetic algorithm presented in this notebook successfully solves the N-Queens problem by evolving a population of potential solutions over several generations. The algorithm's effectiveness is influenced by various factors such as mutation rates, crossover rates, and selection strategies. Through fitness progression plots and visualizations of the chessboard, the algorithm demonstrates its capacity to find conflict-free configurations for boards of different sizes.

The experiments with mutation and crossover rates reveal that careful tuning of these parameters is crucial for balancing the exploration of the search space (mutation) and the exploitation of good solutions (crossover). Elitism proves to be a useful strategy for preserving high-quality solutions, whereas generational strategies may lead to slower convergence or even premature stagnation.